Lecture 4
Processes and Threads

Mingyu Gao
gaomy@tsinghua.edu.cn

IIIS, Tsinghua

Announcement

- HW 1 will be released this week
  - You have about 2 weeks; refer to the deadline on 网络学堂
  - Submit electronically on 网络学堂 or in papers to the TA

- You should been working hard on Lab 1 now. Have fun!

The Need of An Operating System

- So far we have assumed only one program uses processor and memory
- The reality is, each machine should concurrently support multiple users and also multiple programs executed by each user

- Operating system (OS)
  - Control hardware components, e.g., processor, memory, I/O devices
  - Coordinate resource use among multiple programs for multiple users

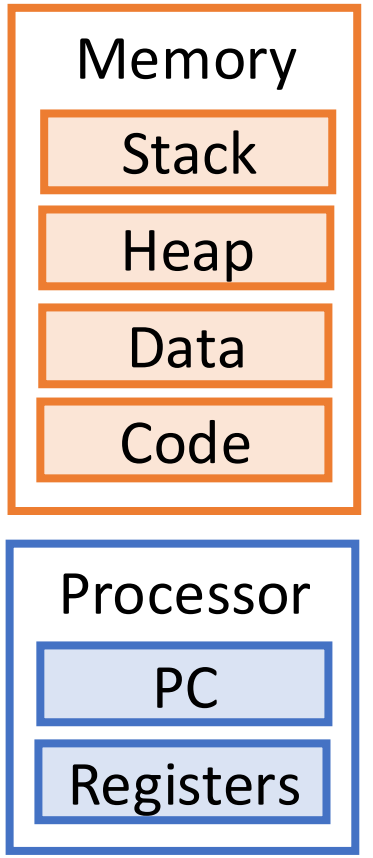
- We will introduce the key concepts in OS in several lectures
  - Process, virtual memory, I/O, file system, protection, …
  - Today: process

Today's Outline

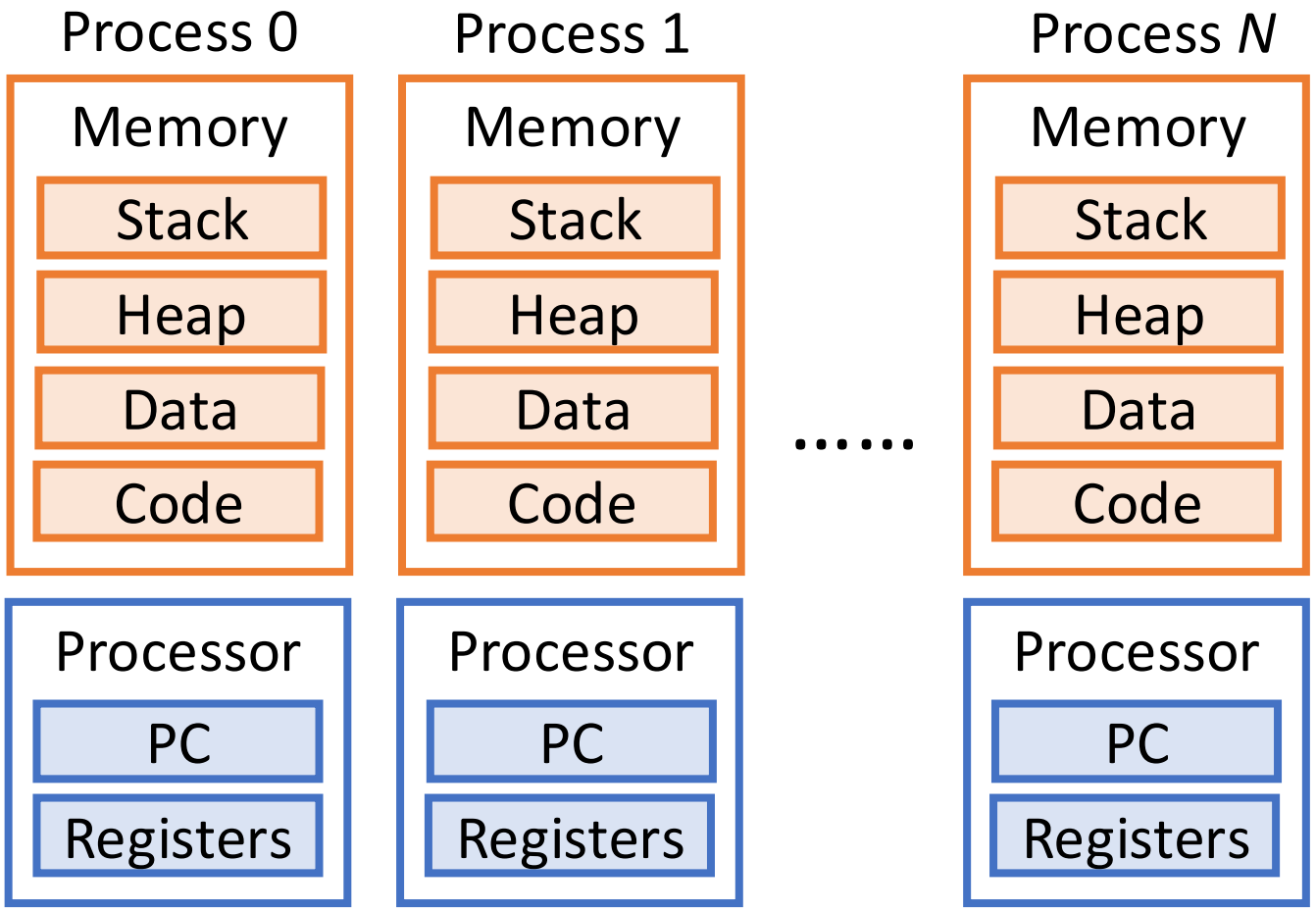
- Processes
  - Threads and address spaces
  - Context switch
  - User mode and kernel mode: system calls, interrupts, exceptions
- Process management
  - Fork, exit, wait, exec
  - Signals
  - Shell
- Scheduling policies
  - FCFS, Round Robin, SJF/SRTF

Processes

- A process is an instance of an executing program
  - Note: to differentiate from “processor” and “program”
- Processes are managed by OS
- Process provides two key abstraction for each program
- Execution context
  - Program seems to have exclusive use of processor
  - With the help of context switch
- Address space
  - Program seems to have exclusive use of memory
  - With the help of virtual memory (later lectures)

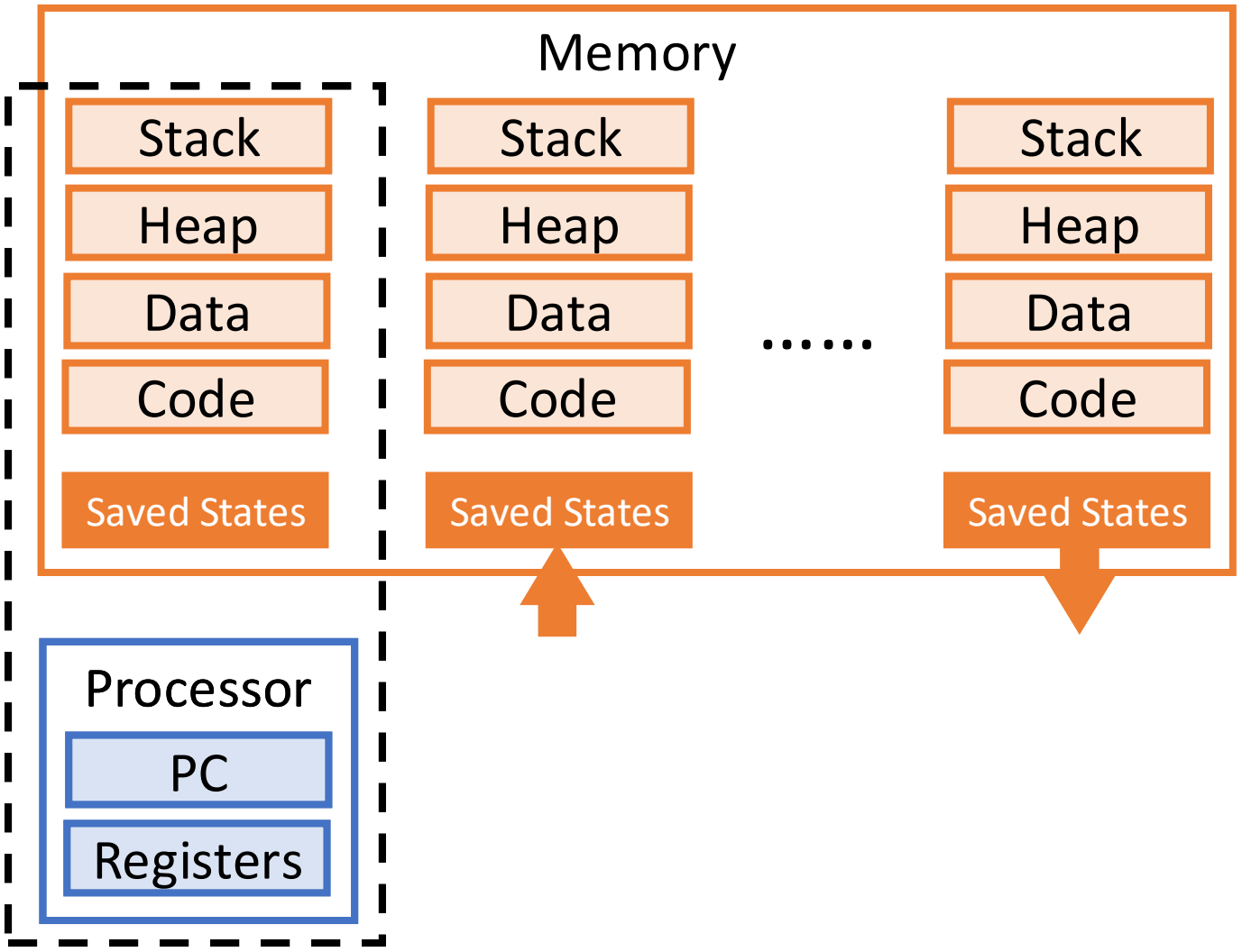


Multiprocessing: The Illusion



- Computer runs many processes simultaneously
  - E.g., applications from one or multiple users, background tasks, …
- Each process seems to have exclusive use of processor and memory

Multiprocessing: The Reality



- Process executions are time-interleaved on the processor
- Address spaces are all mapped to the memory
- States (PC, registers, etc.) of non-executing processes are saved to memory
  - Saved before descheduled
  - Restored before continue
- Multi-core supports concurrent execution

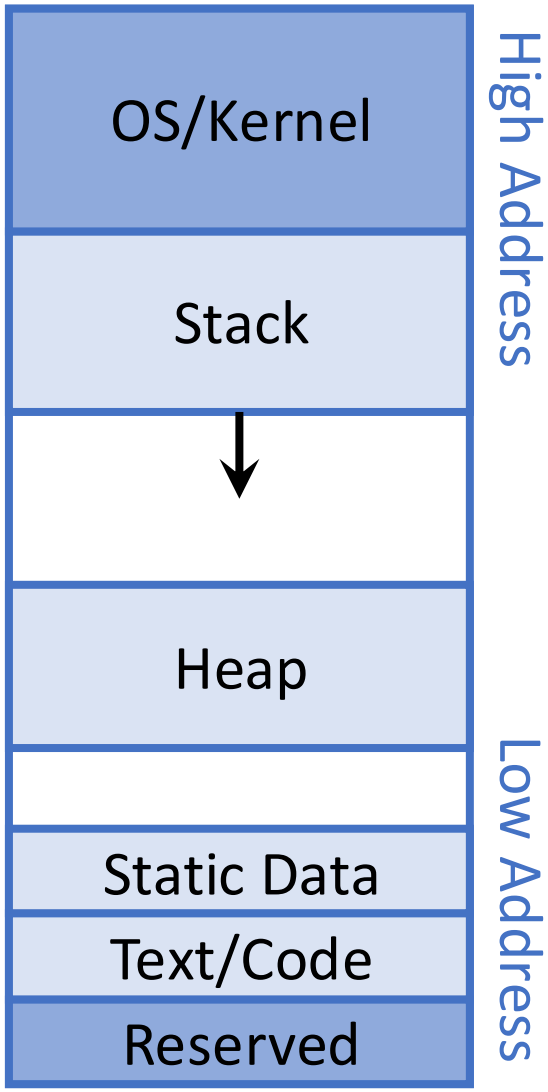
Execution Context: Thread

- A thread is a single unique execution context
  - Including registers, PC, stack pointer, memory, …
- A process can have one or multiple threads (multi-threading programs)

- A thread can be executing, i.e., occupying a processor and running, or suspended, i.e., not executing
- Context switch between threads involves swapping processor states (registers, PC, stack pointer, etc.), but data remain in memory
  - Save those of the current executing thread to memory
  - Restore those of the next executing thread from memory
- Context switch is coordinated by OS

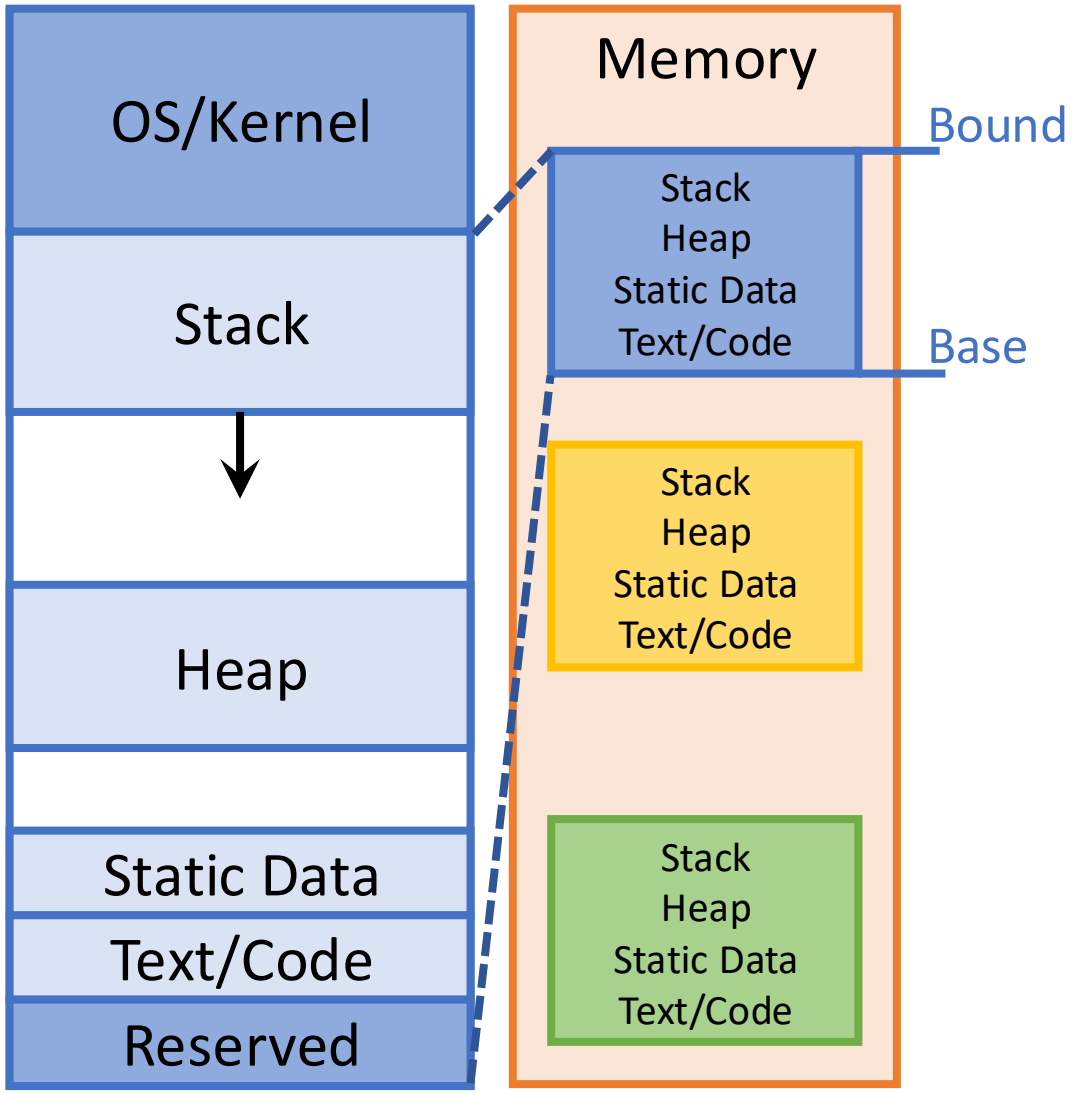
Address Space

- An address space is an illusion of exclusive memory to a program, i.e., the memory layout discussed before
  - OS maps a special region at high address, called kernel, but kernel is inaccessible to user process
- Protection is the main purpose of using address space
  - Different programs must not freely access each other's data
  - A user program must not freely access OS kernel
- Usually one process has one address space, which is shared by all threads of that process
  - Each thread of a process has private registers, PC, stack pointer, …, but they share the same memory content and can access each other's data



Simple Implementation: Base and Bound (B&B)

- Base and bound (B&B)
  - Hardware offers base and bound registers as part of process context
  - OS sets the two registers at context switch
  - Hardware checks the addresses of all memory accesses against them at runtime
- Drawbacks
  - Restricted to contiguous address ranges
  - Difficult to extend dynamically
  - Difficult to support shared regions across address spaces
- Better solution: virtual memory
  - Discussed in later lectures



Process Revisited

- Recall that a process is an instance of a program
- Simply speaking, a process = one or multiple threads + an address space
  - Threads encapsulate concurrency: allow for efficient parallel processing
  - Address spaces encapsulate protection: against buggy or adversarial programs

- Communication is easier within a process, harder between processes
  - Fundamental tradeoff between protection and efficiency
  - Theoretically, parallel processing can also be done using multiple processes, but would be more expensive
- Processes are protected from each other, and OS is protected against user processes

Process Control Block (PCB)

- OS kernel represents each process as a process control block (PCB), an in-memory data structure that contains process states
  - Process ID (PID)
  - Status (executing, ready to execute, blocked, …)
  - Register states
  - Address space states, …
  - User, executable file, priority, …
  - Execution time, …

- The kernel scheduler maintains a data structure containing the PCBs, and a scheduling algorithm selects the next process to execute
  - We discuss scheduling at the end of this lecture

User vs. Kernel Mode

- For the purpose of protection, hardware provides at least two modes
  - Kernel mode (or system mode, supervisor mode, protected mode): high privilege
    - For OS to execute sensitive tasks, e.g., scheduling processes, setting up address spaces
  - User mode: limited privilege
    - For normal user program execution, i.e., a process runs in user mode

- Hardware support
  - One bit in processor (user/kernel mode bit)
  - Certain privileged instructions only permitted in kernel mode
- OS support
  - User to kernel transition: set mode bit; save user process PC; jump to kernel code
    - Save user process PC so that we may return to user process to continue executing
  - Kernel to user transition: clear mode bit; restores user process PC; and continue

Three Ways of User To Kernel Transition

- Transition between the two modes must be well controlled
  - User programs must not freely enter kernel mode
  - Only a few limited mechanisms are allowed, with proper safety guarantees

- System call (syscall): user process purposely requests a system service
  - Like a procedure call but outside user program
  - E.g., process exit, allocate memory, send data to network
- Interrupt: an external event triggers a context switch to kernel
  - Happen asynchronously, e.g., independently of user process
  - E.g., timer, I/O devices (e.g., Ctrl-C)
- Exception and trap: an internal event triggers a context switch to kernel
  - E.g., protection violation (e.g., segfault), divide-by-zero

Comparison of Kernel Transition Mechanisms

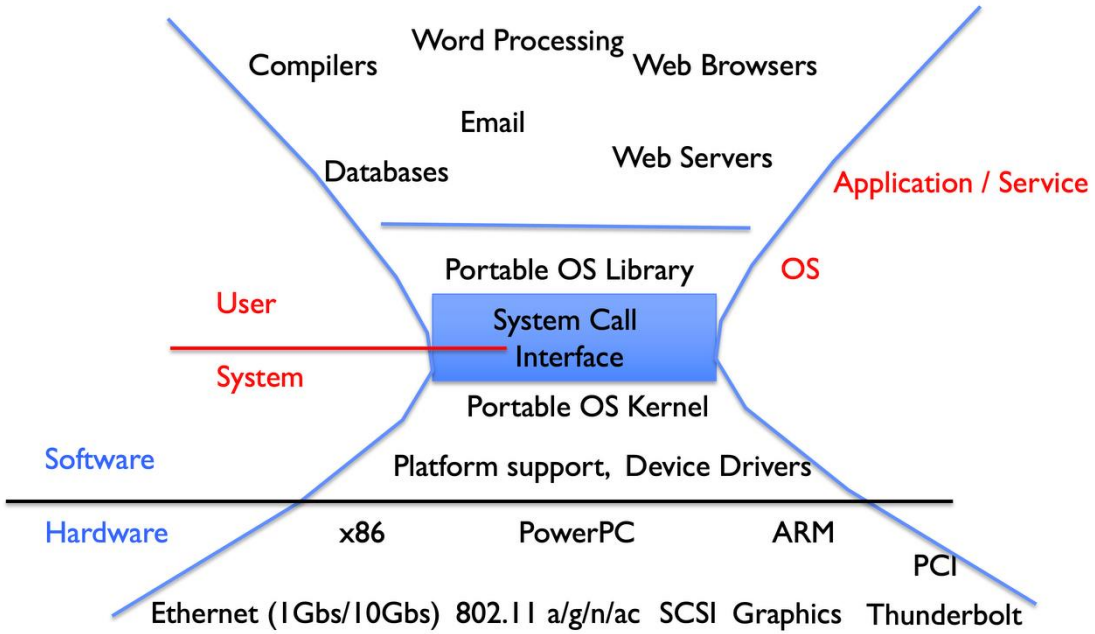
- These three ways are similar to branch/jump and call/return, i.e., also change control flow, but now from user program to kernel code rather than within user program

- Syscalls: proactive (i.e., requested by user program)
  - Implemented similar to procedure calls
  - Program can save and restore PC/registers
- Exceptions: reactive (i.e., unexpected); internal reasons
- Interrupts: reactive; external reasons
  - Not possible for program to know when they may happen, and thus not possible to save and restore PC/registers; need special mechanisms

Syscalls

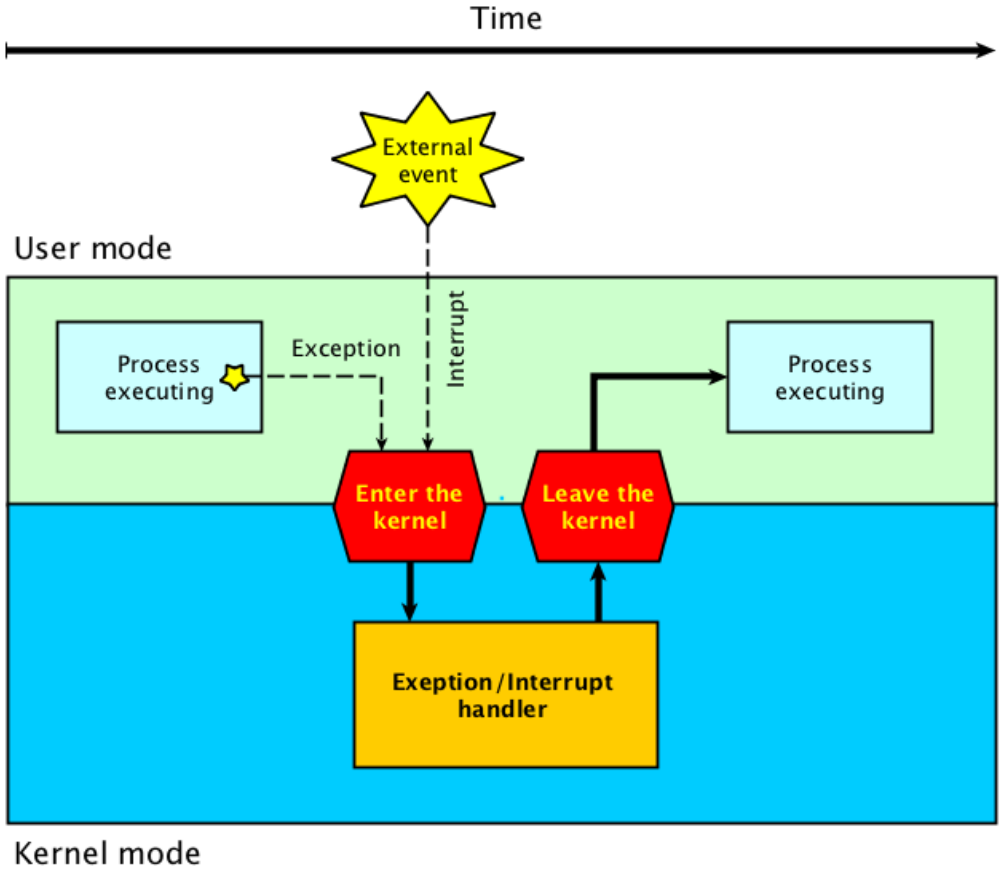
- x86-64 has a `syscall` instruction; RISC-V has an `ecall` instruction
- POSIX Unix defines syscall numbers for different services (over 300)
- Syscalls offer a narrow waist as a portable interface

%eax	Name	Source	%ebx	%ecx	%edx	%esi	%edi
1	sys_exit	kernel/exit.c	int	-	-	-	-
2	sys_fork	arch/i386/kernel/process.c	struct pt_regs	-	-	-	-
3	sys_read	fs/read_write.c	unsigned int	char *	size_t	-	-
4	sys_write	fs/read_write.c	unsigned int	const char *	size_t	-	-
5	sys_open	fs/open.c	const char *	int	int	-	-
6	sys_close	fs/open.c	unsigned int	-	-	-	-
7	sys_waitpid	kernel/exit.c	pid_t	unsigned int *	int	-	-
8	sys_creat	fs/open.c	const char *	int	-	-	-
9	sys_link	fs/namei.c	const char *	const char *	-	-	-
10	sys_unlink	fs/namei.c	const char *	-	-	-	-
11	sys_execve	arch/i386/kernel/process.c	struct pt_regs	-	-	-	-
12	sys_chdir	fs/open.c	const char *	-	-	-	-
13	sys_time	kernel/time.c	int *	-	-	-	-
14	sys_mknod	fs/namei.c	const char *	int	dev_t	-	-
15	sys_chmod	fs/open.c	const char *	mode_t	-	-	-
16	sys_lchown	fs/open.c	const char *	uid_t	gid_t	-	-
18	sys_stat	fs/stat.c	char *	struct __old_kernel_stat *	-	-	-
19	sys_lseek	fs/read_write.c	unsigned int	off_t	unsigned int	-	-
20	sys_getpid	kernel/sched.c	-	-	-	-	-
21	sys_mount	fs/super.c	char *	char *	char *	-	-
22	sys_oldumount	fs/super.c	char *	-	-	-	-
23	sys_setuid	kernel/sys.c	uid_t	-	-	-	-
24	sys_getuid	kernel/sched.c	-	-	-	-	-
25	sys_stime	kernel/time.c	int *	-	-	-	-
26	sys_ptrace	arch/i386/kernel/ptrace.c	long	long	long	long	-
27	sys_alarm	kernel/sched.c	unsigned int	-	-	-	-
28	sys_fstat	fs/stat.c	unsigned int	struct __old_kernel_stat *	-	-	-
29	sys_pause	arch/i386/kernel/sys_i386.c	-	-	-	-	-
30	sys_utime	fs/open.c	char *	struct utimbuf *	-	-	-



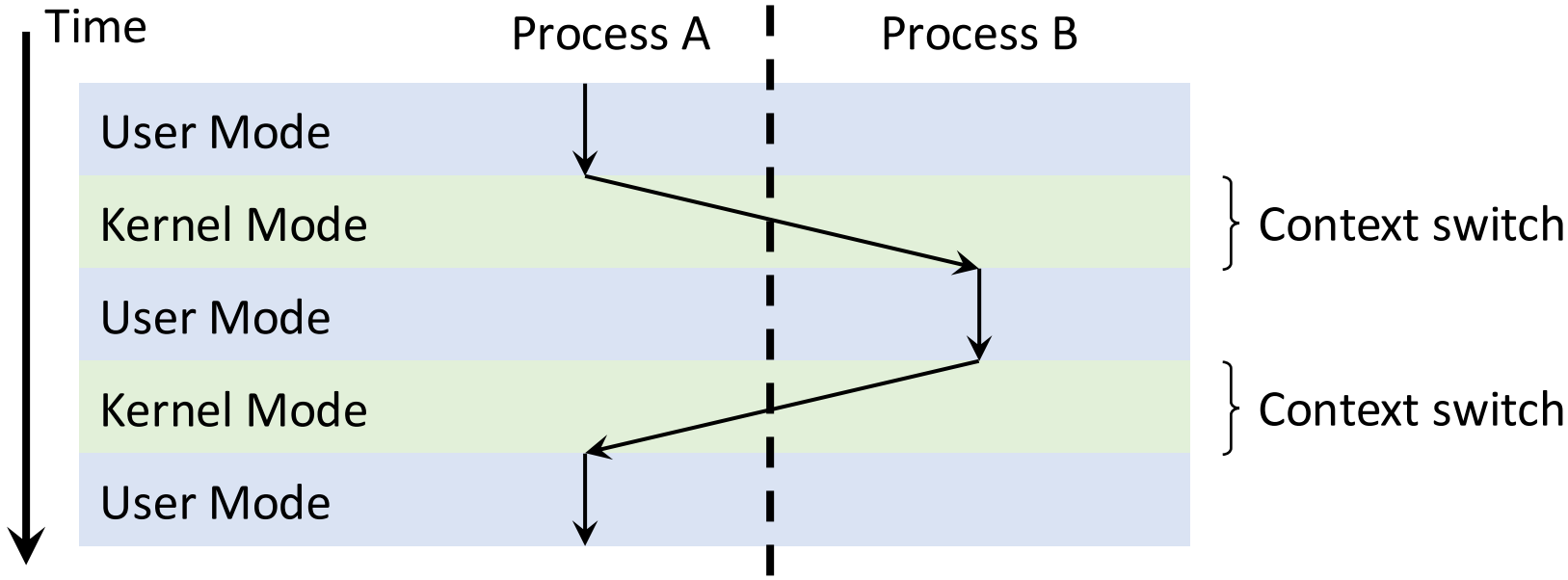
Exceptions and Interrupts

- Despite of different causes, OS handles exceptions and interrupts similarly
  - An internal exception occurs, or an external interrupt arrives
  - Processor stops the user process, saves current PC and the cause, and transfers control flow to the kernel
  - Exceptions/interrupts are processed by kernel exception handler, which is selected based on the cause, to fix problems or handle events
  - Return to user process to continue (the current instruction or the next instruction); or abort if problems cannot be fixed

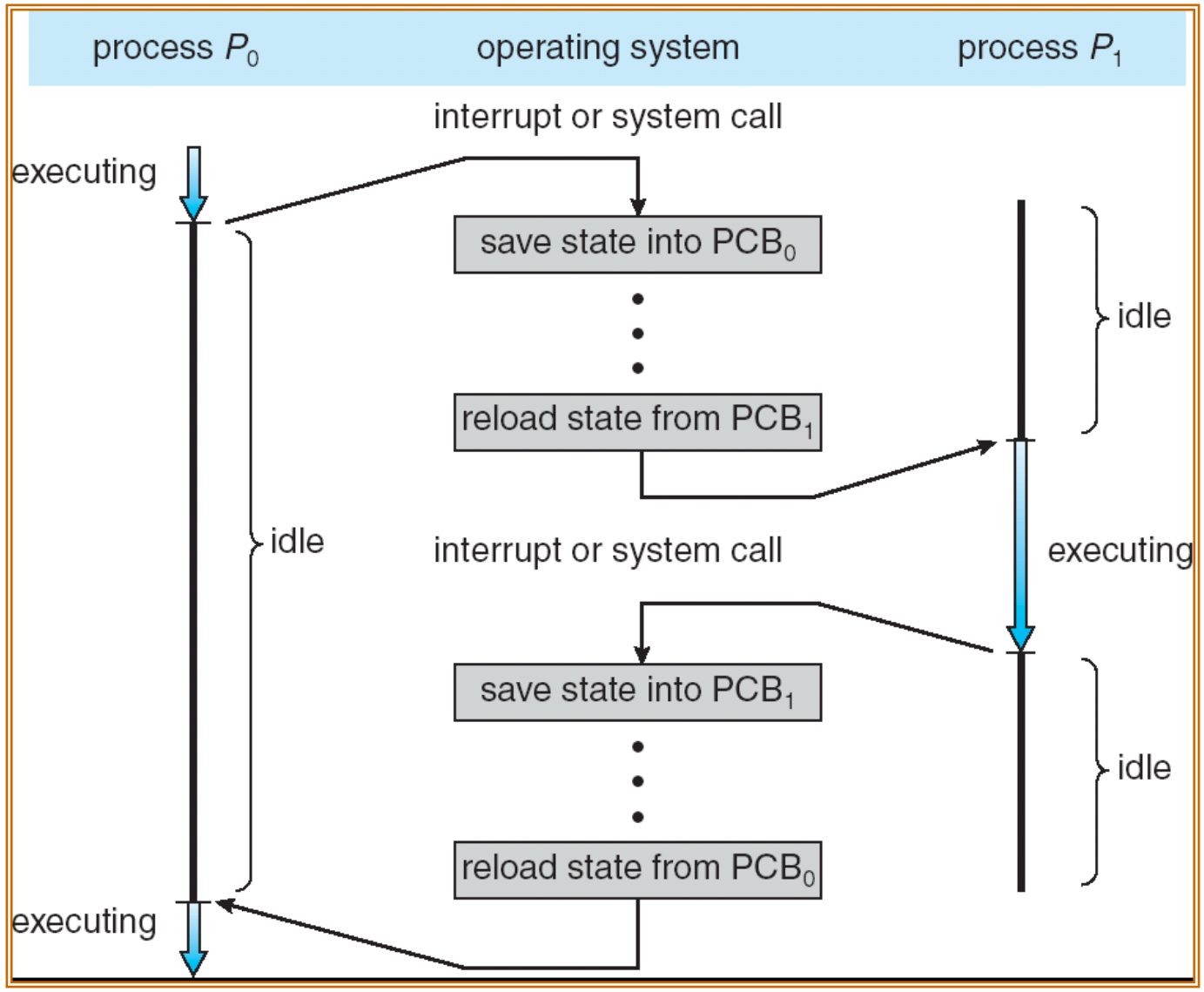


Context Switch Revisited

- Recall that context switch is handled by OS kernel; this means it must use the above three mechanisms to transit from user to kernel
- Context switch could be triggered by
  - An interrupt of a timer, e.g., every 100 microseconds
  - A syscall to voluntarily yield the processor, e.g., sleep



Context Switch Details



Summary

- Process: an instance of a running program
  - One or multiple parallel threads + protected address space
- Thread: execution context
  - Fully describes program execution states: register, PC, stack pointer, memory, …
- Address space: memory addresses accessible to program
  - Isolate processes from each other; isolate OS from processes

- User mode vs. kernel mode
  - Three ways of transition: syscalls, interrupts, exceptions

Today's Outline

- Processes
  - Threads and address spaces
  - Context switch
  - User mode and kernel mode: system calls, interrupts, exceptions
- Process management
  - Fork, exit, wait, exec
  - Signals
  - Shell
- Scheduling policies
  - FCFS, Round Robin, SJF/SRTF

Syscalls for Process Management

- `exit` – terminate a process
- `fork` – copy the current process
- `exec` – change the program being executed by the current process
- `wait` – wait for a process to finish
- Signals – send and receive messages between processes

Terminating Processes

- A process is terminated, i.e., stopped permanently, for 1 of 3 reasons
  - Returning from `main` function
  - Calling `exit` syscall
  - Receiving a signal whose default action is to terminate (discussed soon)

- `void exit(int status)`
  - Terminates with an exit status of `status`
  - Exit status can also be set by `main` function return value
  - Convention: normal return status is 0; non-zero indicates errors
- `exit` is called but never returns (to the program)

Creating Processes

- Parent process creates a new running child process by calling `fork`
- Child process is almost identical (i.e., a copy) to parent process
  - Child gets an identical (but separate) copy of the parent's address space
  - Child gets identical access permissions of the parent's I/O devices
  - Child has a different PID than the parent
- Parent and child processes execute concurrently
  - Cannot predict their execution order; may interleave in any way

- `int fork(void)`
  - Return 0 to child process; return child's PID to parent process
  - Or return negative value to indicate errors
- `fork` is called once but returns twice

Fork Example

```
pid_t mypid;
pid_t pid = getpid();       // current processes PID
int x = 0;
printf("Parent pid [%d], x: %d\n", pid, x);

pid_t cpid = fork();

if (cpid > 0) {             // Parent process
  mypid = getpid();
  printf("[%d] parent of [%d], x: %d\n", mypid, cpid, ++x);
} else if (cpid == 0) {     // Child process
  mypid = getpid();
  printf("[%d] child, x: %d\n", mypid, --x);
} else {                    // Error
  perror("Fork failed");
}
```

Q: What does this print?

```
Parent pid [1111], x: 0
[1111] parent of [2222], x: 1
[2222] child, x: -1
```

Or

```
Parent pid [1111], x: 0
[2222] child, x: -1
[1111] parent of [2222], x: 1
```

- Cannot predict execution order
- Duplicate but separate address spaces; subsequent changes to x are independent

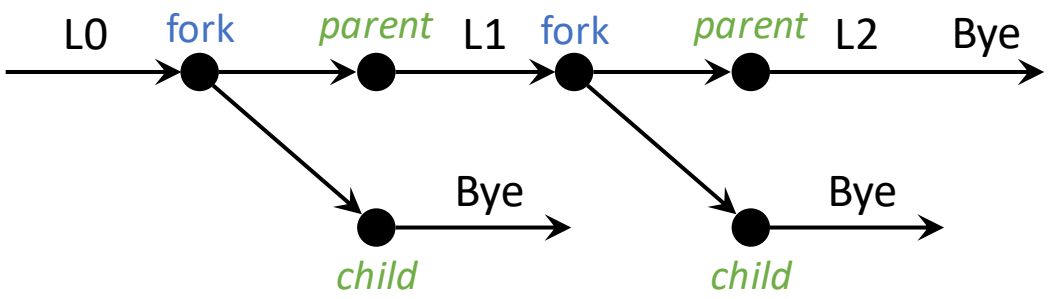
Nested Forks in Parent

```
printf("L0\n");

if (fork() != 0) {
  printf("L1\n");

  if (fork() != 0) {
    printf("L2\n");
  }
}

printf("Bye\n");
```



- Are the following outputs feasible?

L0	L0
L1	Bye
Bye	L1
Bye	Bye
L2	Bye
Bye	L2

  - Parent prints L0, L1, L2, Bye in order
  - First child prints Bye after L0
  - Second child prints Bye after L1

Reaping Child Processes

- After a process terminates, it still occupies system resources (e.g., exit status, various OS structures), and becomes a zombie process
  - Because parent may need to investigate exit status later
- Parent can reap its children with `wait` or `waitpid`
  - Parent is given the exit status information, and then kernel deletes zombie child

- `pid_t wait(int* wstatus)`
  - Suspends current process until one of its children terminates
  - Return value is the PID of the child process that terminated
  - If `wstatus` != NULL, then store the child exit status into that address
- `pid_t waitpid(pid_t pid, int* wstatus, int options)`
  - Suspends current process until the specific child process `pid` terminates
  - Various options (see Linux man page)

Wait Examples

```
int child_status;
if (fork() == 0) {
  printf("Hello from child\n");
  exit(0);
} else {
  printf("Hello from parent\n");
  int cpid = wait(&child_status);
  printf("Child %d has terminated\n", cpid);
}
```

Q: What could be the sequence of output?

If reaping with multiple children, the children may terminate in arbitrary order

```
for (i = 0; i < N; i++) {
  if ((pid[i] = fork()) == 0) exit(100+i);   // Child
}
// Parent
for (i = 0; i < N; i++) {
   pid_t wpid = wait(&child_status);
   if (WIFEXITED(child_status))
     printf("Child %d terminated with exit status %d\n", wpid, WEXITSTATUS(child_status));
   else
     printf("Child %d terminate abnormally\n", wpid);
}
```

What If Parents Do Not Reap?

- If any parent terminates without reaping a child, then the orphaned child is reaped by the special `init` process (whose PID is 1)
  - So only need explicit reaping in long-running processes, e.g., shells and servers

```
if (fork() == 0) {        // Child
  printf("Terminating Child, PID = %d\n", getpid());
  exit(0);
} else {                  // Parent
  printf("Running Parent, PID = %d\n", getpid());
  while (1) {}            // Infinite loop
}
```

Zombie child when parent does not terminate. If we kill parent, child is also reaped by `init`.

```
if (fork() == 0) {        // Child
  printf("Running Child, PID = %d\n", getpid());
  while (1) {}            // Infinite loop
} else {                  // Parent
  printf("Terminating Parent, PID = %d\n", getpid());
  exit(0);
}
```

Non-terminating child, i.e., child process is still active though parent has terminated. Must explicitly kill child (then it will be reaped by `init`), otherwise will keep running indefinitely.

Loading and Running Programs

- exec family: load and run in the current process
- `int execve(char* filename, char* argv[], char* envp[])`
  - With the executable `filename`, argument list `argv`, and env variable list `envp`
    - By convention `argv[0] == filename`
    - `envp` has the format of `name=value` strings (e.g., `USER=joe`)
  - Retain PID, but overwrite code, data, and stack
- See man page for other variants: `execle`, `execvp`, `execvpe`, …
  - Different on how to provide argument list, env var list, …
- `execve` is called once and never returns, except if there is an error
- The common style: fork-then-exec

Exec Example

Generic fork-then-exec style

```
pid = fork();
if (cpid == 0) {
  // Child loads and runs new program
  exec(...);
} else if (cpid > 0) {
  // Parent waits for child
  wait(...);
}
```

Full example

```
pid_t cpid = fork();
if (cpid == 0) {            // Child Process
  // Child loads and runs new program
  char *args[] = {"ls", "-l", NULL};
  execv("/bin/ls", args, NULL);
  // execv only returns if failed
  perror("execv");
  exit(1);
} else if (cpid > 0) {      // Parent Process
  // Parent waits for child
  pid_t tcpid = wait(&status);
}
```

Signals

- Signal: small message to notify a process that some event has occurred
  - Similar to interrupts and exceptions
  - Sent from the kernel (sometimes at the request of another process) to a process
  - Signal type is identified by integer-typed signal IDs (about 30 types)

Signal ID	Name	Default Action	Event
2	SIGINT	Terminate	Ctrl-C
9	SIGKILL	Terminate	Kill program
11	SIGSEGV	Terminate	Segmentation violation
14	SIGALRM	Terminate	Timer
17	SIGCHLD	Ignore	Child terminated

Sending and Receiving Signals

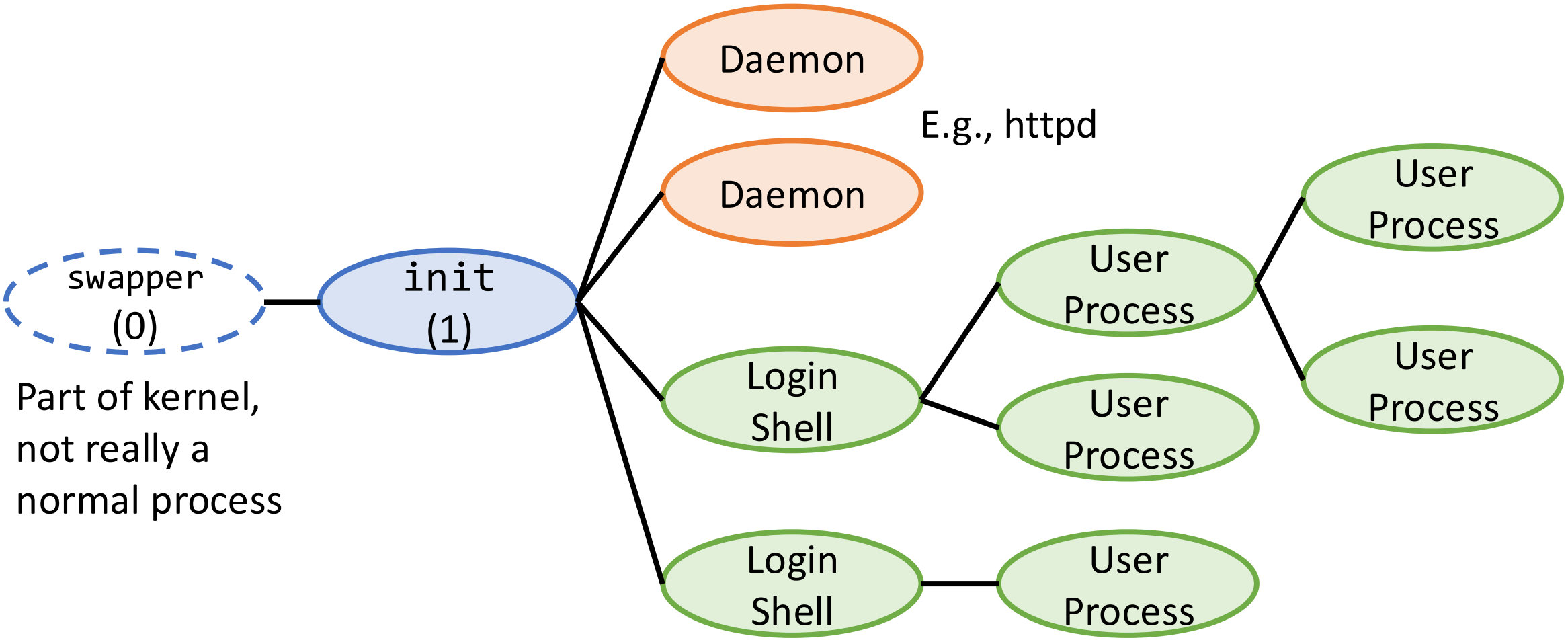
- Sending a signal
  - Kernel detects a system event, e.g., termination of a child process (SIGCHLD)
  - Or, another process calls the `kill` syscall to request kernel to send a signal
    - `int kill(pid_t pid, int sig)`
    - Note: `kill` can send various signals, not just SIGKILL

- Receiving a signal
  - Terminate the process (optionally with core dump)
  - Or, execute a user-level function called signal handler
    - Similar to exception handlers
    - User can register a signal handler through `sigaction` syscall
  - Or, ignore the signal (do nothing)

Shell

- A shell is a job control system that runs programs on behalf of user
- Example shells
  - `sh`: original Unix shell (Stephen Bourne, AT&T Bell Labs, 1977)
  - `csh/tcsh`: BSD Unix C shell
  - `bash`: "Bourne-Again" shell (default Linux shell)
- Shell is a process itself: fork itself, exec program, and wait (or not)
  - Foreground jobs: shell waits and reaps
  - Background jobs: not reaped by shell, and also not reaped by init because shell (typically) will not terminate
    - Use signals to manage
    - Or, bring background jobs to foreground and then reap

Linux Process Hierarchy



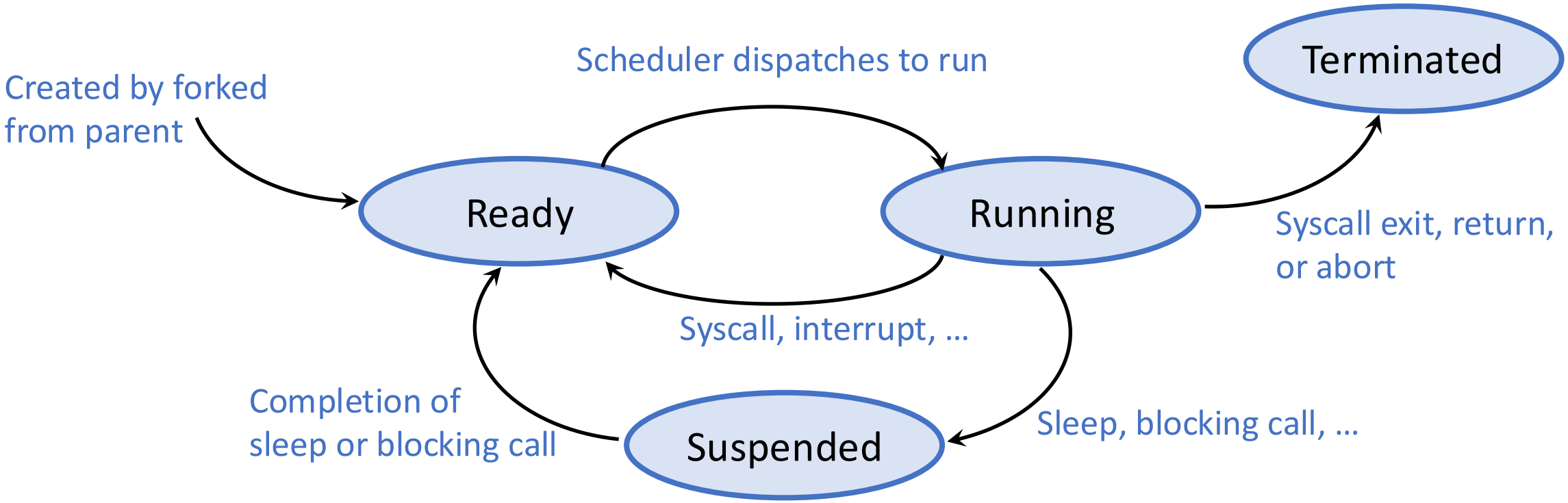
Summary

- Spawning processes: `fork`
  - One call, two returns (to parent and to child)
- Terminating processes: `exit`
  - One call, no return
- Waiting for and reaping child processes: `wait`, `waitpid`
- Loading and running programs: `exec` family, e.g., `execve`
  - One call, no return if succeed
- Sending/receiving signals: `kill`, `sigaction`

Today's Outline

- Processes
  - Threads and address spaces
  - Context switch
  - User mode and kernel mode: system calls, interrupts, exceptions
- Process management
  - Fork, exit, wait, exec
  - Signals
  - Shell
- Scheduling policies
  - FCFS, Round Robin, SJF/SRTF

Recall: Life Cycle of Processes/Threads



Scheduling

- When there are more threads than hardware cores, active threads reside in a ready queue, waiting to be put onto the cores to run
- OS decides which threads to run first, using a scheduling policy
- Scheduling also applies to other scenarios
  - E.g., serving user requests on cloud, executing instructions on superscalar cores



Simplifications and Assumptions

- We use a simple job model to study scheduling, i.e., there are multiple jobs that would like to finish on a single processor
  - Jobs are independent, no ordering constraints
  - Jobs can be paused and resume later; and generate results at the end
  - Jobs have different execution time
  - Jobs are equal, no priorities

- Two classes
  - Non-preemptive: once giving a job some period of time to execute (not necessarily to completion), cannot take processor back before the period ends
  - Preemptive: we can take processor back at any time, and use it for other jobs; preempted job goes back to ready queue and can resume later

Scheduling Metrics

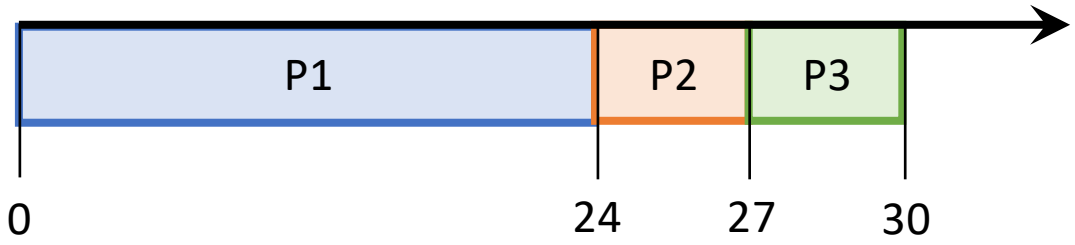
- Waiting time: time when the job waits in the ready queue
- Response time = waiting time + execution time
  - This is what the user sees
- Throughput: number of jobs completed per unit of time
  - Related to response time, but not always the same thing
- Fairness: all jobs use resource in some equal way

- To minimize response time? For one job or average of all jobs?
- To maximize throughput?
- To maximize fairness?

FCFS (First-Come, First-Served)

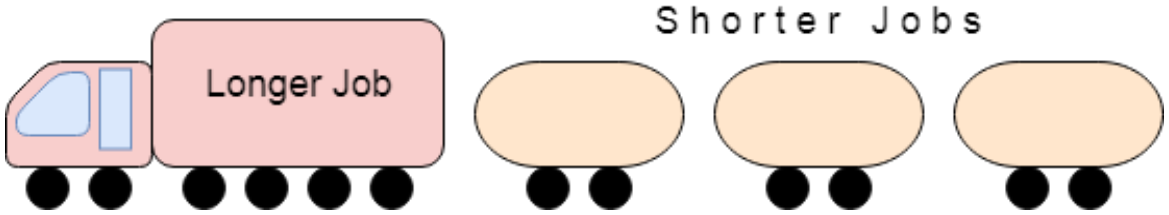
- FCFS, a.k.a., “first in, first out” (FIFO): run until done, non-preemptive
- Example: suppose jobs arrive in the order P1, P2, P3

Job	Exe Time
P1	24
P2	3
P3	3



  - Wait time: P1 = 0, P2 = 24, P3 = 27; average wait time = 17
  - Average response time = (24+27+30)/3 = 27

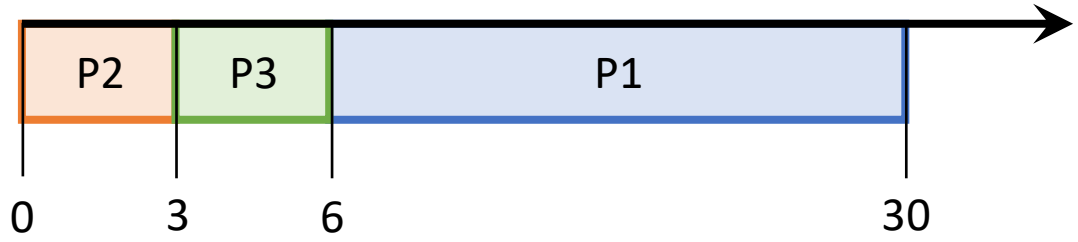
- Convoy effect: short jobs are blocked behind long jobs



FCFS

- Same example, but now suppose jobs arrive in the order P2, P3, P1

Job	Exe Time
P1	24
P2	3
P3	3



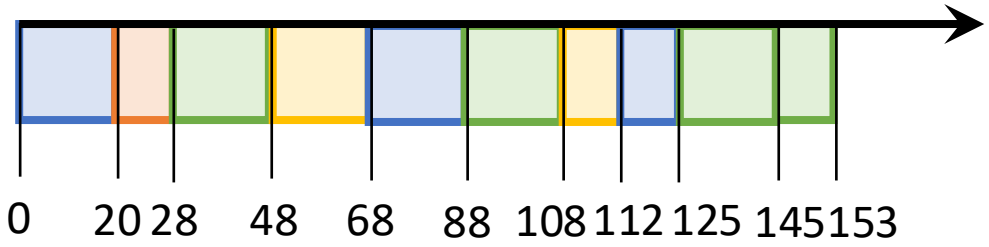
  - Average wait time = (0+3+6)/3 = 3 << 17, much better than before
  - Average response time = (3+6+30)/3 = 13 << 27, much better than before

- FCFS summary
  - ✓ Simple
  - ✗ Depend on arrival orders
  - ✗ Average wait/response time suffers from Convoy effect

Round Robin (RR)

- Not to depend on job arrival order
- Round Robin: each job gets a small unit of time (time quantum, q) to execute, and then gets preempted and added back to end of queue
- Example: with a time quantum q = 20, and zero context switch cost

Job	Exe Time	Remaining Time
P1	53	33 13 0
P2	8	0
P3	68	48 28 8 0
P4	24	4 0



  - Wait time: P1 = (68–20)+(112–88) = 72, P2 = 20, P3 = 85, P4 = 88
  - Average wait time = 66.25
  - Average response time = (125+28+153+112)/4 = 104.5

Round Robin

- Choices of time quantum (a.k.a., time slice)
  - Large q: approach FCFS, may cause Convoy effect
    → Average wait/response time suffers
  - Small q: preempt too often, and suffer from context switch overheads
    → Throughput suffers
  - Typically choose q so that context switch overhead is small enough (e.g., 5%)

- Round Robin summary
  - ✓ More fair for short jobs than FCFS
  - ✗ Context switch adds overheads
  - ✗ Not easy to choose the right time quantum value
  - Q: Assuming no context switch cost, is RR always better than FCFS in terms of average response time?

Optimal Scheduling: SJF/SRTF

- What if we know precisely job execution time *before* scheduling?
  - Usually not possible when executing user processes or serving user requests
- SJF (shortest job first): run the job with the shortest execution time first
- SRTF (shortest remaining time first): preemptive version of SJF
  - If new job arrives and has shorter remaining time than current one, preempt
  - Also called STCF (shortest time to completion first)
- SJF/SRTF is the optimal policy to minimize average response time

- SJF/SRTF summary
  - ✓ Optimal average response time
  - ✗ Unfair, may lead to starvation if many short jobs come in; long jobs never run!
  - ✗ Difficult to predict job execution time; sometimes impractical

Summary

- Processes
  - Threads and address spaces
  - Context switch
  - User mode and kernel mode: system calls, interrupts, exceptions
- Process management
  - Fork, exit, wait, exec
  - Signals
  - Shell
- Scheduling policies
  - FCFS, Round Robin, SJF/SRTF

Next Lecture:
Practical Skills (By Prof. Xu)

See you in 4 weeks after midterm!